Information Technology/AI23231/36

Ex No: 3a

IMPLEMENTATION OF BLOCKS WORLD PROGRAM

Aim:

To implement binary search tree.

Procedure:

Program:

```
# Class representing the state of the blocks world
class BlocksWorld:
def __init__(self, num_blocks):
# Initialize blocks world with num_blocks on the table
self.state = [[block] for block in range(num_blocks)] # Each block starts on the table
self.num_blocks = num_blocks
def display_state(self):
# Display the current state of the blocks world

for stack in self.state:
print(f"Block(s) on stack: {stack}")
def move(self, block, destination):
# Move block onto another block or the table
# Find the block in the current state and move it
source_stack = self.find_block(block)

destination_stack = self.find_block(destination)
if source_stack is None or destination_stack is None:
print(f"Invalid block {block} or destination {destination}.")
return
# Remove the block from its current position
```

```
source_stack.remove(block)
# Put the block on the destination stack
destination_stack.append(block)
# Display the state after the move
self.display_state()
def find_block(self, block):
# Find the stack that contains the given block
for stack in self.state:
if block in stack:
return stack
return None
def goal_state(self, goal):
# Goal state is a list of stacks that represents the desired configuration
self.state = goal
print("Goal state set.")
self.display_state()
# Main program to solve Blocks World problem
def main():
# Initialize the Blocks World with 3 blocks (0, 1, 2)

blocks_world = BlocksWorld(3)
print("Initial state:")
blocks_world.display_state()
# Define the goal state (Block 0 is on top of Block 1, and Block 2 is on the table)
goal = [[0, 1], [2]] # Goal configuration
blocks_world.goal_state(goal)
print("\nPerforming Moves:")
# Move Block 0 onto Block 2
```

```
blocks_world.move(0, 2)
# Move Block 1 onto Block 2
blocks_world.move(1, 2)
# Move Block 2 onto the table (empty stack)
blocks_world.move(2, 0)
# Run the main program
if __name__ == "__main__":
main()
```

Output:

```
Initial state:
Block(s) on stack: [0]
Block(s) on stack: [1]
Block(s) on stack: [2]
Goal state set.
Block(s) on stack: [0, 1]
Block(s) on stack: [2]
Performing Moves:
Block(s) on stack: [1]
Block(s) on stack: [2, 0]
Block(s) on stack: []
Block(s) on stack: [1]
Block(s) on stack: [2, 0]
Block(s) on stack: []

Block(s) on stack: [1, 2]
Block(s) on stack: [0]
Block(s) on stack: []
```

IMPLEMENTION OF A FUZZY INFERENCE SYSTEM

Aim:

To implement Fuzzy Inference System.

Procedure:

Program:

Example: Simple Fuzzy Inference System for Temperature Control

```
import numpy as np
import skfuzzy as fuzz
import matplotlib.pyplot as plt
# Step 1: Define the fuzzy variables (input: temperature, output: fan speed)
temperature = np.arange(0, 41, 1) # Temperature from 0 to 40 degrees Celsius
fan_speed = np.arange(0, 11, 1) # Fan speed from 0 to 10
# Step 2: Define fuzzy membership functions
# Temperature membership functions: Low, Medium, High
temp_low = fuzz.trimf(temperature, [0, 0, 20])
temp_medium = fuzz.trimf(temperature, [10, 20, 30])

# Fan speed membership functions: Low, Medium, High
fan_low = fuzz.trimf(fan_speed, [0, 0, 5])
fan_medium = fuzz.trimf(fan_speed, [2, 5, 8])
fan_high = fuzz.trimf(fan_speed, [5, 10, 10])
# Step 3: Visualize the membership functions
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(temperature, temp_low, label='Low')
```

```
plt.plot(temperature, temp_medium, label='Medium')
plt.plot(temperature, temp_high, label='High')
plt.title("Temperature Membership Functions")
plt.xlabel("Temperature (°C)")
plt.ylabel("Membership Degree")
plt.legend()

plt.subplot(2, 1, 2)
plt.plot(fan_speed, fan_low, label='Low')
plt.plot(fan_speed, fan_medium, label='Medium')
plt.plot(fan_speed, fan_high, label='High')
plt.title("Fan Speed Membership Functions")
plt.xlabel("Fan Speed")
plt.ylabel("Membership Degree")
plt.legend()
plt.tight_layout()
plt.show()

# Step 4: Fuzzification - Define a crisp temperature input
temperature_input = 28 # Example temperature
# Fuzzify the input
temp_low_level = fuzz.interp_membership(temperature, temp_low, temperature_input)
temp_medium_level = fuzz.interp_membership(temperature, temp_medium,
temperature_input)
temp_high_level = fuzz.interp_membership(temperature, temp_high, temperature_input)
# Step 5: Apply fuzzy rules
# Rule 1: If temperature is Low, fan speed is Low
# Rule 2: If temperature is Medium, fan speed is Medium
```

```
# Rule 3: If temperature is High, fan speed is High
fan_low_level = temp_low_level
fan_medium_level = temp_medium_level
fan_high_level = temp_high_level
# Step 6: Defuzzification - Calculate the crisp output (fan speed)
fan_output = fuzz.defuzz(fan_speed, fan_low * fan_low_level + fan_medium *
fan_medium_level + fan_high * fan_high_level, 'centroid')
# Step 7: Print the output
print(f"Temperature: {temperature_input}°C")
print(f"Fuzzified fan speed: {fan_output:.2f}")
```

Output

```
Temperature: 28°C
Fuzzified fan speed: 6.00
```